

Now to add a new type of Entity object, all we need to do is create a new class that extends Entity and provide an implementation for Move. This is a win for extensibility we do not have to modify the Entity class to add additional classes. However, if we want to add new functionality to Entity (e.g., a Draw method), we need to add that method to Entity and each class that derives from Entity.

Being an object oriented (OO) language steeped in the C-tradition, C# tackles these problems in the same way as Java. However, it does so in a more refined way with particular emphasis on common problems encountered with Java. Keep in mind these properties as you encounter new features in C#'s object system and think about how they enforce these properties (or not!).

2.2 The basics of C#

Syntactically, the basics of C# are derived from the C++ way of doing OO rather than Java. So the syntax may look unfamiliar if you have not programmed in C++, but the basic ideas are the same. Like Java, C# is a single-inheritance language with a unified type system. That is, all types derive from a common class, `System.Object` in both cases. Here is a series of Java class definitions and the equivalent in C#.

Java Implementation:

```
public abstract class Employee {
    private int yearsEmployed;
    public Employee(int yearsEmployed) {
        this.yearsEmployed = yearsEmployed;
    }

    public int getYearsEmployed() { return
yearsEmployed; }
    public String getTitle();
}

public class Tech extends Employee
    private String speciality;
    public Tech(string speciality, int yearsEmployed)
{
    super(yearsEmployed);
    this.speciality = speciality;
}
```